ARCHITECTURE & DESIGN SPECIFICATION · v2

GSIP

The Grounded Simulation-in-the-loop Platform

Provenance-gated simulation-in-the-loop optimisation. The language model frames the problem, routes it, and explains the result — but is architecturally forbidden from supplying the facts that decide the answer.

Prepared for **Sumukh Guruprasad**

6 August 2026

Architecture & Design Specification (v2)

> Lineage note: this is the successor to the original *GSIP enhanced pipeline* design. The acronym is re-read from "Generic/Distributed Simulation & Optimisation Platform" to **Grounded Simulation-in-the-loop Platform**, because the central design commitment of v2 is *provenance discipline* — the system is architecturally forbidden from treating model-recalled values as ground truth.

---

0. What changed in one paragraph

v1 established a genuinely good instinct: **all result numbers come from deterministic simulation code, never from the LLM.** v2 keeps that intact and closes the hole it left open — that the *inputs* to the simulator were still being invented by the LLM and passed off as fact. v2 does this with four new load-bearing components: a **problem classifier** that routes questions to curated framing playbooks (and can abstain), a **parameter-triage stage** that identifies which inputs actually determine the answer, a **provenance model** that tags every value with where it came from, and a **provenance gate** that refuses to run — or loudly flags — when a decision-critical input is backed only by the model's memory. A **fidelity descriptor** on each domain pack stops a toy model from being dressed up as a validated one.

Table of Contents

---

1. Design goals and non-goals

Goals

Goal	Meaning
Grounded truth	Every value that materially affects an answer is traceable to a real source (user, dataset, instrument), never to model memory.
Honest confidence	The confidence a result is *presented* with matches the evidence and model fidelity behind it.
Wide applicability	The platform frames and attempts a broad range of problems — but knows, and states, where it has no validated model.
Reproducibility	Same run spec → same scenarios → same numbers.
Auditability	Every decision, assumption, and value provenance is logged in a run ledger.
Discovery, not just confirmation	Optimisation explores beyond the model's conventional priors.

Non-goals

- **Not** a system that produces numeric facts from an LLM. The LLM frames, routes, questions, proposes, and explains. It never *is* a measurement.
- **Not** a claim to reason "from first principles" in the philosophical sense. It applies *curated, classified framings* — textbook relations and characteristic questions — which is honest and useful, but is retrieval-of-structure, not derivation-from-axioms.
- **Not** a universal solver. Coverage is bounded by the domain packs and playbooks that exist; the system surfaces that boundary rather than hiding it.

2. Core principles — the trust model

Five principles, ordered. Lower-numbered principles win conflicts.

1. **Provenance before precision.** A value's *source* is checked before its *number* is used. A precise-looking number with no grounding is treated as less trustworthy than a rough number from a real source.
2. **Non-negotiable truth (preserved from v1).** All outcome numbers are produced by deterministic simulation code. The LLM never emits a result quantity.
3. **Fidelity honesty.** A model's output is presented with the fidelity tier of the model that produced it. A toy model yields *illustrative* output, never *predictive* output, no matter how fluent the surrounding prose.
4. **Competence boundaries are explicit.** The classifier and domain packs can say "I have no validated frame/model for this." Abstention is a first-class output, not a failure.
5. **Determinism and audit.** Reproducible runs; every AI output, assumption, provenance tag, and gate decision logged.

The crucial reframe from v1: **reproducible ≠ correct.** A deterministic simulator computing faithfully on a hallucinated input gives the *same wrong answer every time*. Determinism protects the arithmetic; only provenance protects the truth.

3. System architecture — the ten stages

The Grounded Pipeline — end-to-end workflow

The three jobs to keep separate in your head:

- **Classifier + playbooks** govern *what you need to know* (the shape of the unknowns).
- **Triage** governs *which of those actually matter* (the sensitivity map).
- **Provenance gate** governs *where each value is allowed to come from* (the trust wall).

---

4. The provenance model (core innovation)

Every value used anywhere downstream is not a bare number — it is a `ParameterValue` carrying its origin and uncertainty.

Provenance taxonomy

Provenance	Meaning	Trust class
`USER_SUPPLIED`	The user stated it.	**Grounded**
`MEASURED`	From an instrument / sensor reading.	**Grounded**
`RETRIEVED`	From a named dataset, API, or tool (carries `source_id`).	**Grounded**
`COMPUTED`	Derived deterministically from grounded inputs.	**Grounded** (inherits weakest parent)
`DEFAULT`	A domain-pack default / documented assumption.	**Ungrounded (declared)**
`LLM_RECALL`	The model produced it from training memory.	**Ungrounded (unverified)**

Key distinction: `RETRIEVED` is grounded not because it is *guaranteed true* but because it is *attributable and checkable* — retrieval ≠ truth, but it converts an untraceable guess into a traceable claim someone can verify or contest. `LLM_RECALL` is the only class that is both unverified *and* unattributable, which is why it is treated as the floor.

`COMPUTED` inherits the **weakest** provenance among its inputs: a value computed partly from an `LLM_RECALL` input is itself effectively ungrounded, and the gate treats it that way. This prevents laundering an ungrounded value into a grounded-looking one by passing it through arithmetic.

Every value carries uncertainty

A `ParameterValue` also carries an uncertainty estimate. Ungrounded values are assigned wide priors by default; grounded values carry the uncertainty of their source (a sensor's error bar, a dataset's stated confidence). This feeds output confidence bands in stage 10 (see §7 for how triage sensitivities turn input uncertainty into output uncertainty cheaply).

---

5. The provenance gate

This is the component that actually fixes v1's central flaw. It sits between grounding and simulation and combines two axes: **how much a parameter matters** (from triage, §7) and **how well-grounded it is** (from the provenance model, §4).

The gating matrix

Criticality ↓ / Provenance →	Grounded	Default (declared)	LLM-recall (unverified)
Decision-critical	Run	Flag + require user confirmation	**Block** (or explicit, logged override)
Moderate	Run	Run + log assumption	Flag + log
Low	Run	Run	Run + log

The single rule that matters: **a decision-critical parameter may never silently take an LLM-recalled value.** The system must either ground it (retrieve / ask the user / measure) or refuse. If the user insists, the override is explicit, logged, and stamps the entire result as ungrounded.

This is what stops the Delhi failure mode: the LLM asserting "Anand Vihar at (45, 80), intensity 0.9" is an `LLM_RECALL` value, and source location/intensity are decision-critical for a dispersion problem, so the gate blocks the silent run and forces grounding.

Gate outputs

Each gate decision is a `ProvenanceGateDecision` written to the ledger: the parameter, its criticality, its provenance, the verdict (`RUN` / `FLAG` / `BLOCK`), and — for blocks — the clarification or retrieval action triggered.

---

6. Problem classification subsystem

Classifier

A small model (fine-tuned SLM or LLM) that maps a question to problem *types*. Design requirements:

- **Multi-label, not single-label.** Real problems occupy several frames at once. "Reduce Delhi pollution" is *simultaneously* spatial-dispersion, policy-feasibility, economic, and public-health. A single label silently discards the frames that might hold the real answer. Output is a ranked set of `(type, confidence)` pairs.
- **Calibrated confidence.** Confidences must mean something (temperature-scaled / isotonic-calibrated), because thresholds downstream depend on them.
- **Abstention / open-set detection.** If no type clears a confidence threshold, or the input is out-of-distribution, the classifier returns `ABSTAIN`. The system then either runs in an explicit *generic mode* with heightened flagging, or asks the user to characterise the problem. This is the make-or-break capability for a "solve anything" ambition: without it, "solve any problem" degrades into "confidently mis-frame any problem."
- **Frame disambiguation.** When multiple frames score highly and imply different answers, surface the competition to the user ("this is both a dispersion and a policy problem — which do you care about, or

both?") rather than picking one.

Playbooks

Each problem type maps to a **curated playbook** — the reusable problem-solving method for that class (this is the CommonKADS / "generic tasks" idea made concrete). A playbook contains:

Field	Contents
`required_factors`	The variables this class of problem always turns on.
`characteristic_questions`	The questions an expert asks for this class.
`candidate_models`	Governing models / domain packs applicable to this class.
`sensitivity_priors`	Prior expectation of which factors are decision-critical (seeds triage).
`known_pitfalls`	Documented ways this class of problem misleads.
`applicability_bounds`	Where this framing stops being valid.

Honesty about where playbooks come from: they are curated artifacts, not free. If the LLM writes them unchecked, you have merely relocated recall-with-confidence up one level. The recommended path is *LLM-drafts / human-ratifies / versioned*: the model proposes a playbook, a human with domain knowledge edits and signs off, and the playbook is version-controlled. This makes the curation cost explicit and keeps a human in the loop for the framing knowledge — which is exactly the knowledge that must be trustworthy.

---

7. Parameter triage and sensitivity

Triage answers: *of everything the playbook says we need, which values actually move the answer?* This is the classifier's highest-value payoff — it turns the framing into a **map of where truth is expensive.**

Two sources of sensitivity

1. **Prior sensitivities** (cheap, static): from the playbook's `sensitivity_priors`. Available immediately, no computation.
2. **Empirical sensitivities** (cheap, dynamic): a fast screening sweep on the domain pack at its cheapest fidelity — **Morris elementary-effects screening** (one-at-a-time perturbations across the feasible space). Morris is chosen because it identifies which parameters are influential at a fraction of the cost of a full variance-based (Sobol) analysis, which matters when each simulation call costs money/time.

The empirical pass matters because it replaces *LLM-asserted* importance with *measured* importance — the model's opinion about what matters is itself just recall, and here we can cheaply check it against the actual simulator.

Output

A `TriageResult`: every required parameter ranked by decision-criticality (`CRITICAL` / `MODERATE` / `LOW`), which then (a) sets the strictness of the provenance gate per parameter, and (b) orders clarification so we spend the user's patience on the three parameters that decide the answer, not the twenty that don't.

Bonus: cheap output-uncertainty

Because triage already computes each parameter's sensitivity `S_i`, and the provenance model already attaches an input uncertainty `σ_i` to each value, output uncertainty can be approximated cheaply by propagating `S_i · σ_i` contributions — no separate Monte-Carlo pass required. A decision-critical parameter backed only by a wide default produces a visibly wide output band. This is how stage 10's confidence bands stay honest for free.

---

8. Clarification and grounding

Grounding resolves each required parameter to a `ParameterValue` with an explicit provenance. Clarification is the *human-facing* part of grounding, and it is now **triage-driven**: it asks first (and sometimes only) about the parameters that are both decision-critical and not yet grounded.

This fixes v1's "clarification theatre," where the checklist asked easy knowns (generator type, budget) while the determinative unknowns stayed guessed. Now:

- **Critical + ungrounded** → asked, or routed to retrieval, before anything runs.
- **Moderate + ungrounded** → asked if cheap, else defaulted with a logged assumption.
- **Low** → defaulted silently, logged.

Each clarification question declares the **provenance it will produce**: a question answered by the user yields `USER_SUPPLIED`; a "point me to your dataset" prompt yields `RETRIEVED`; the system never satisfies a critical parameter by having the LLM fill it in.

---

9. Scenario generation

Scenario generation proposes the candidate configurations the optimiser will search. v2 keeps AI seeding but fixes the discovery-suppression problem from v1 drawback #5.

Split-budget generation

Track	Fraction (tunable)	Source	Purpose
Exploit	~60%	AI-proposed candidates, grounded in playbook + real context	Relevance — realistic, plausible configurations
Explore	~40%	Space-filling (LHS / Sobol) across the *full* feasible region, un-fenced by AI priors	Discovery — the non-obvious regions where novel optima hide

The explore track is deliberately *not* seeded from the LLM's suggestions, because the LLM's priors are conventional by construction (they are the internet's consensus), and fencing the search to them structurally biases the system toward conventional answers dressed as insight — self-defeating for an optimiser whose

point is discovery. Compare FunSearch: it works precisely because the evaluator is ground truth and the search is allowed to wander.

Scenario values also pass the gate

Any *specific value* in an AI-proposed scenario (a source coordinate, an intensity) is `LLM_RECALL` and is tagged as such. If a scenario's defining value is decision-critical and ungrounded, it is flagged the same way any other value would be — AI-proposed scenarios get no exemption from the provenance gate.

---

10. Domain packs and the fidelity descriptor

The domain-pack contract from v1 is preserved and extended. Domain expertise still lives *only* in the pack; the platform stays generic.

Preserved contract

```
DomainPackBase
├── state_schema()      → Pydantic model for initial conditions
├── action_schema()     → Pydantic model for actions/levers
├── simulate()          → Runs the simulation, returns OutcomeBundle
├── score()             → Computes metrics from outcome
├── feasibility()       → Checks if state+actions are valid
└── cost_model()        → Estimates compute cost per fidelity level
```

New: fidelity descriptor

Every pack must declare what kind of model it is. This is what prevents confidence-laundering (v1 drawback #2) — a fluent natural-language wrapper can no longer make a toy read like a validated study.

Field	Values / meaning
`fidelity_tier`	`TOY` · `REDUCED_ORDER` · `VALIDATED` · `CALIBRATED`
`validation_status`	`UNVALIDATED` · `BACKTESTED` · `EXPERT_REVIEWED` · `GROUND_TRUTH_CALIBRATED`
`applicability_bounds`	Regimes / ranges where the model is valid
`known_limitations`	Explicit list of what the model omits or simplifies
`reference`	Citation / provenance of the model itself

The explanation layer (§12) is **required** to propagate this. A `TOY` / `UNVALIDATED` pack produces output labelled *illustrative, not predictive*, regardless of how specific the framing was. The Delhi advection-diffusion pack is `REDUCED_ORDER` + `UNVALIDATED` against real monitoring data, so its numbers are explicitly illustrative — they show *relative* effects of interventions, not forecast concentrations.

New: uncertainty-aware outputs

`simulate()` returns an `OutcomeBundle` that carries not just point metrics but uncertainty (error bars / distributions where the pack supports it; the §7 sensitivity-propagated band otherwise). Scoring and the Pareto front carry this uncertainty forward so "best" is never reported as a bare point when the inputs don't justify it.

---

11. Simulation, scoring, optimisation

Unchanged in spirit from v1, and deliberately so — this is the part that was already right.

- **Simulation:** the domain pack runs deterministic code. Numbers come from code, not AI.
- **Scoring:** pure, deterministic math (rubrics, benchmarks). No AI in scoring.
- **Optimisation:** Bayesian optimisation for sample-efficient search over expensive simulations; NSGA-II for multi-objective / Pareto-front problems. Iterates until the compute budget (from `cost_model()`) is exhausted or convergence is reached.

The only addition: optimisation is **uncertainty-aware** where the pack provides it — dominated-solution checks on the Pareto front account for output uncertainty so the front isn't falsely precise.

---

12. Confidence-calibrated explanation

The LLM writes the human-readable report, and it is bound by two hard rules that make the confidence of the *prose* match the evidence:

1. **Fidelity propagation.** The report states the model's fidelity tier and validation status up front. Illustrative results are called illustrative. There is no fluent phrasing that upgrades a `TOY` result to a prediction.
2. **Provenance disclosure.** The report surfaces which decision-critical inputs were grounded vs defaulted vs overridden. If the run proceeded on an ungrounded critical input via explicit override, the report leads with that caveat.

The report also links results back to the playbook framing and explains *why* the top solutions win in terms of the governing factors — but always inside the confidence envelope the evidence supports.

---

13. Data structures

```
from enum import Enum
from typing import Any, Optional
from pydantic import BaseModel


# ── Provenance ──────────────────────────────────────────────────────

class Provenance(str, Enum):
    USER_SUPPLIED = "user_supplied"   # grounded
    MEASURED      = "measured"        # grounded
    RETRIEVED     = "retrieved"       # grounded (attributable)
    COMPUTED      = "computed"        # grounded (inherits weakest parent)
    DEFAULT       = "default"         # ungrounded, declared
    LLM_RECALL    = "llm_recall"      # ungrounded, unverified  ← the floor

GROUNDED = {Provenance.USER_SUPPLIED, Provenance.MEASURED,
            Provenance.RETRIEVED, Provenance.COMPUTED}


class ParameterValue(BaseModel):
    """A value is never bare — it carries origin and uncertainty."""
    name: str
    value: Any
```

```
    provenance: Provenance
    source_id: Optional[str] = None      # dataset/API/sensor id for RETRIEVED/MEASURED
    uncertainty: Optional[float] = None  # std / half-width; wide default if ungrounded
    parents: list[str] = []              # for COMPUTED: names of inputs it derives from
    notes: Optional[str] = None


# ─── Classification & playbooks ──────────────────────────────────────────

class ProblemTag(BaseModel):
    type: str                            # e.g. "spatial/dispersion"
    confidence: float                    # calibrated


class ProblemClassification(BaseModel):
    tags: list[ProblemTag]               # multi-label, ranked
    abstained: bool = False              # open-set: no validated frame
    generic_mode: bool = False           # proceed generically with heightened flags


class Playbook(BaseModel):
    problem_type: str
    version: str                         # curated + versioned; human-ratified
    required_factors: list[str]
    characteristic_questions: list[str]
    candidate_models: list[str]          # domain packs applicable to this class
    sensitivity_priors: dict[str, float] # factor → prior criticality
    known_pitfalls: list[str]
    applicability_bounds: list[str]


# ─── Triage ──────────────────────────────────────────────────────────────

class Criticality(str, Enum):
    CRITICAL = "critical"
    MODERATE = "moderate"
    LOW      = "low"


class ParameterCriticality(BaseModel):
    name: str
    criticality: Criticality
    sensitivity: float                   # empirical (Morris) or prior
    source: str                          # "morris_screening" | "playbook_prior"


class TriageResult(BaseModel):
    ranked: list[ParameterCriticality]


# ─── Clarification ───────────────────────────────────────────────────────

class ClarificationQuestion(BaseModel):
    id: str
    parameter: str                       # which value this grounds
    question: str
    why_needed: str
    produces_provenance: Provenance      # what grounding this yields
    options: Optional[list[str]] = None
    is_required: bool                    # true if the parameter is CRITICAL
    default_if_skipped: Optional[Any] = None


# ─── Domain-pack fidelity ────────────────────────────────────────────────
```

```
class FidelityTier(str, Enum):
    TOY           = "toy"
    REDUCED_ORDER = "reduced_order"
    VALIDATED     = "validated"
    CALIBRATED    = "calibrated"


class ValidationStatus(str, Enum):
    UNVALIDATED             = "unvalidated"
    BACKTESTED              = "backtested"
    EXPERT_REVIEWED         = "expert_reviewed"
    GROUND_TRUTH_CALIBRATED = "ground_truth_calibrated"


class DomainPackFidelity(BaseModel):
    fidelity_tier: FidelityTier
    validation_status: ValidationStatus
    applicability_bounds: list[str]
    known_limitations: list[str]
    reference: Optional[str] = None

    @property
    def is_predictive(self) -> bool:
        return self.fidelity_tier in {FidelityTier.VALIDATED, FidelityTier.CALIBRATED}


# ─── The gate ──────────────────────────────────────────────────────────

class GateVerdict(str, Enum):
    RUN   = "run"
    FLAG  = "flag"
    BLOCK = "block"


class ProvenanceGateDecision(BaseModel):
    parameter: str
    criticality: Criticality
    provenance: Provenance
    verdict: GateVerdict
    action: Optional[str] = None         # e.g. "trigger_clarification:q3"
    override: bool = False               # true if user forced a blocked run


def gate(pc: ParameterCriticality, pv: ParameterValue) -> ProvenanceGateDecision:
    grounded = pv.provenance in GROUNDED
    if pc.criticality == Criticality.CRITICAL:
        if grounded:
            verdict = GateVerdict.RUN
        elif pv.provenance == Provenance.DEFAULT:
            verdict = GateVerdict.FLAG
        else:  # LLM_RECALL
            verdict = GateVerdict.BLOCK
    elif pc.criticality == Criticality.MODERATE:
        verdict = GateVerdict.RUN if grounded or pv.provenance == Provenance.DEFAULT \
                  else GateVerdict.FLAG
    else:  # LOW
        verdict = GateVerdict.RUN
    return ProvenanceGateDecision(
        parameter=pc.name, criticality=pc.criticality,
        provenance=pv.provenance, verdict=verdict,
    )
```

```
# ── Enriched state & outcomes ──────────────────────────────────────────

class EnrichedState(BaseModel):
    values: dict[str, ParameterValue]    # every field carries provenance
    subject: Optional[str] = None
    context_summary: str
    assumptions: list[str]


class OutcomeBundle(BaseModel):
    metrics: dict[str, float]
    uncertainty: dict[str, float]        # per-metric band
    fidelity: DomainPackFidelity         # travels WITH the numbers


class ConfidenceReport(BaseModel):
    fidelity: DomainPackFidelity
    grounded_critical: list[str]
    ungrounded_critical: list[str]       # non-empty ⇒ result is caveated/illustrative
    overrides: list[str]
    output_bands: dict[str, float]
    verdict: str                         # "predictive" | "illustrative" | "blocked"
```

14. End-to-end worked examples

Example A — “How to reduce pollution in Delhi?” (the v1 failure case, now handled)

2. Classification (multi-label): `spatial/dispersion (0.82)`, `policy/feasibility (0.55)`, `public-health (0.41)`. Frames disagree on the answer → surface to user: *optimise dispersion outcomes, policy-feasible interventions, or a weighted mix?* User picks dispersion + feasibility.

3. Playbook (spatial/dispersion): - required_factors: source locations, source intensities, wind field, diffusion rate, grid bounds, sink/mitigation zones - sensitivity_priors: source locations & intensities → high; wind → high; diffusion → medium; bounds → low

4. Triage (Morris screening on cheap-fidelity pack): | Parameter | Criticality | Source | |————|————-|———| | source intensities | CRITICAL | morris | | source locations | CRITICAL | morris | | wind field | CRITICAL | morris | | diffusion rate | MODERATE | morris | | grid bounds | LOW | prior |

5. Grounding & clarification: - Emission inventory (locations + intensities) → attempt `RETRIEVED` from a named source (e.g. a CPCB/monitoring dataset the user provides). If unavailable → clarification asks the user for the data or a dataset link. **The LLM is never allowed to fill these in.** - Wind field → `RETRIEVED` from a weather dataset, or `USER_SUPPLIED` seasonal assumption logged as such. - Diffusion rate → `DEFAULT` from the pack, logged.

6. Provenance gate: - Suppose no emission dataset is provided and the LLM “knows” the hotspots. Those values are `LLM_RECALL` + `CRITICAL` → **BLOCK**. The run does not proceed on invented coordinates. The user is asked to supply or point to real data, or to explicitly override (which stamps the whole result *illustrative*). - With a real inventory supplied → `RETRIEVED` + `CRITICAL` → RUN.

8–10. Simulate / score / explain: - Pack fidelity: `REDUCED_ORDER` + `UNVALIDATED` → report states results are **illustrative** — they rank interventions by *relative* effect, and do **not** forecast concentrations. - Confidence report lists which critical inputs were grounded, the output bands, and the illustrative verdict.

Contrast with v1, which would have silently run on the LLM's invented hotspots and produced a fluent, authoritative-sounding "Delhi pollution study" built on fabricated inputs.

Example B — "How to increase this generator's output?"

- **Classify:** `energy/electromechanical (0.88)`.
- **Playbook:** governing relations `P = V·I`, `V = N·dΦ/dt`; factors: flux, turns, RPM, wire resistance, cooling, load.
- **Triage:** RPM and flux → CRITICAL; cooling → MODERATE (caps current); air-gap → MODERATE.
- **Grounding:** generator type, current RPM, current output → `USER_SUPPLIED` via clarification (these are the *determinative* unknowns, so they're asked first, not buried under "what's your budget?"). Material limits → `RETRIEVED` from spec sheet or `DEFAULT` with logged assumption.
- **Gate:** current RPM is CRITICAL; if the user can't supply it and the LLM guesses → BLOCK until grounded.
- **Fidelity:** if the pack is a first-order lumped model → `REDUCED_ORDER`; report frames results as directional guidance, not a design guarantee.

---

15. Service and file layout

```
services/orchestrator/
├── models/
│   ├── provenance.py            # Provenance, ParameterValue, gate()
│   ├── classification.py        # ProblemClassification, ProblemTag
│   ├── playbook.py              # Playbook (loaded from versioned registry)
│   ├── triage.py                # Criticality, TriageResult
│   ├── clarification.py         # ClarificationQuestion, responses
│   ├── fidelity.py              # DomainPackFidelity, tiers
│   └── outcomes.py              # OutcomeBundle, ConfidenceReport, EnrichedState
├── activities/
│   ├── classify_problem.py      # SLM/LLM classifier + abstention
│   ├── retrieve_playbook.py     # playbook registry lookup
│   ├── triage_parameters.py     # Morris screening on cheap fidelity
│   ├── ground_and_clarify.py    # resolve values, tag provenance, ask user
│   ├── provenance_gate.py       # run/flag/block decisions
│   ├── propose_scenarios.py     # exploit track (grounded AI seeds)
│   ├── expand_scenarios.py      # explore track (LHS/Sobol, un-fenced)
│   └── explain_results.py       # confidence-calibrated report
├── workflows/
│   └── run_workflow.py          # Temporal workflow; human-in-loop pause at gate
├── packs/                       # domain packs (each declares fidelity)
│   ├── base.py                  # DomainPackBase + fidelity requirement
│   ├── dispersion/
│   └── electromechanical/
├── registry/
│   └── playbooks/               # versioned, human-ratified playbook YAML/JSON
└── ledger/
    └── run_ledger.py            # append-only audit of every decision + provenance
```

- **Orchestration:** Temporal workflow (`run_workflow.py`) with a human-in-the-loop pause at the provenance gate and at clarification.
- **Parallelism:** Ray for scenario/simulation fan-out.
- **Retrieval store:** Milvus (or equivalent) for grounding sources and playbook similarity.

- **API:** `POST /api/runs`, `POST /api/runs/{id}/clarify`, `POST /api/runs/{id}/override` (explicit, logged gate override).

16. Implementation plan

Phase	Deliverable	Key files
1. Provenance core	`Provenance`, `ParameterValue`, `gate()`, ledger integration. Everything else depends on this.	`models/provenance.py`, `ledger/`
2. Fidelity descriptor	Extend `DomainPackBase`; retrofit existing packs with fidelity metadata; enforce propagation to reports.	`models/fidelity.py`, `packs/base.py`
3. Classifier + playbooks	SLM/LLM classifier with calibration + abstention; playbook registry; LLM-draft/human-ratify authoring flow.	`activities/classify_problem.py`, `registry/playbooks/`
4. Triage	Morris screening on cheap fidelity; criticality ranking; wire to gate strictness.	`activities/triage_parameters.py`
5. Grounding + gate	Triage-driven clarification; provenance tagging; gate verdicts; human-in-loop pause + override endpoint.	`activities/ground_and_clarify.py`, `activities/provenance_gate.py`
6. Split-budget scenarios	Exploit (grounded AI seeds) + explore (un-fenced LHS/Sobol); scenario values pass the gate.	`activities/propose_scenarios.py`, `expand_scenarios.py`
7. Confidence reporting	Fidelity + provenance propagation into the explanation; output bands via triage sensitivities.	`activities/explain_results.py`
8. Calibration hooks	Where ground truth exists, compare sim vs reality; record in a validation register; upgrade `validation_status`.	`packs/*/validation/`

Phase 1 first, always — the provenance core is the spine everything else hangs on.

17. What changed from v1 and why

#	v1 drawback	v2 fix	Where
1	Trust boundary in the wrong place — inputs were LLM-hallucinated	Provenance model + provenance gate; critical + LLM-recall ⇒ blocked	§4, §5
2	Confidence–fidelity mismatch — toy models read as validated	Fidelity descriptor propagated into every report; toy ⇒ illustrative	§10, §12

#	v1 drawback	v2 fix	Where
3	“First principles” overclaim — really recall + templating	Reframed as curated, classified playbooks; abstention when no valid frame	§1, §6
4	Clarification theatre — asked easy knowns, not determinative ones	Triage-driven clarification targets critical ungrounded params first	§7, §8
5	AI priors fenced the search toward conventional answers	Split-budget generation with an un-fenced explore track	§9
6	Closed epistemic loop; coverage illusion	Fidelity/validation status is explicit; calibration hooks; abstention surfaces the coverage boundary	§10, §16

Preserved from v1 because they were already right: numbers-from-code-only, reproducibility, audit trail, deterministic scoring, generic-platform / domain-pack split.

18. Open problems and honest limitations

This design closes v1’s main hole but does not pretend to be finished. The remaining hard parts, stated plainly:

1. **Open-set classification is genuinely hard.** Reliable abstention — knowing when a problem falls outside every validated frame — is an unsolved research problem, not a threshold you tune once. A mis-abstention (running generically when a real frame existed) or a false-confident classification both degrade the system. This is the riskiest single component.
2. **Playbook curation cost and staleness.** Human-ratified playbooks are trustworthy but expensive to author and maintain, and they go stale. The system’s real coverage grows only as fast as playbooks are curated — the “solve anything” front-end will always be able to *frame* more than the back-end can *validly answer*.
3. **Retrieval ≠ truth.** Grounding to a dataset makes a value attributable and checkable, not correct. A wrong-but-cited emission inventory still produces wrong answers — now with a citation. Provenance raises the floor; it does not guarantee the ceiling.
4. **Sensitivity screening has its own cost.** Morris screening is cheap relative to Sobol, but for genuinely expensive simulators even the screening sweep is non-trivial, and prior-only triage is weaker. There’s a real cost/accuracy trade in the triage stage itself.
5. **Fidelity tiers are coarse.** “REDUCED_ORDER” spans a huge range of actual trustworthiness. The tiering is honest signalling, not a precise error bound, and depends on whoever authored the pack rating it truthfully.
6. **Ground-truth calibration is often unavailable.** The calibration hooks (Phase 8) only fire where real outcomes exist. For many interesting problems (long-horizon, counterfactual, one-shot) there is no ground truth to calibrate against, so those packs stay `UNVALIDATED` — correctly labelled, but still unvalidated.

The honest one-line summary: v2 makes the system *trustworthy about its own trustworthiness* — it will tell you when it doesn’t know, refuse to invent the values that matter, and label illustrative results as illustrative. That is a real and defensible advance over v1. It is not the same as being right, and the design does not claim to be.